

SmartTutor AI

Full-Stack Architecture Document

PROJECT NAME	SmartTutor AI – Personalized Learning & Doubt-Solving Agent
TEAM NAME	AI Innovators
DOCUMENT VERSION	v1.0.0 (Production Blueprint)
CORE ECOSYSTEM	Mastra Orchestrator Qdrant Cloud Enkrypt AI

1. Architecture Overview

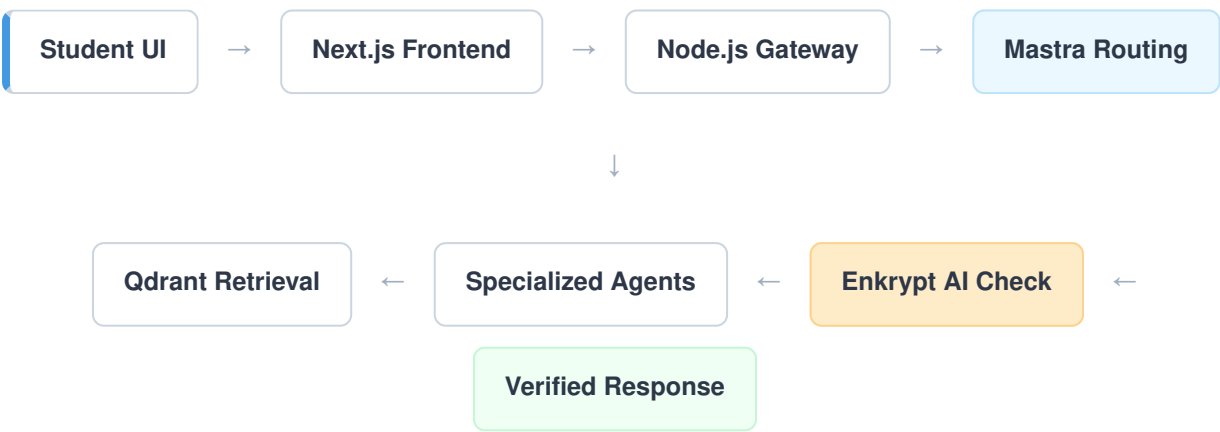
SmartTutor AI is a production-grade multi-agent learning platform engineered to provide deeply personalized tutoring, adaptive synchronous quizzes, dynamic study roadmaps, and persistent long-term learning memory. The fundamental thesis of this system is to bridge the gap left by generic conversational LLMs, which lack specialized workflow steering, structural memory retention, and institutional educational safety frameworks.

The engineering layout leverages an event-driven, decoupled multi-agent paradigm structured explicitly around three mandatory technological backbones:

- **Mastra:** Serves as the central intelligence orchestrator, state machine, and agent coordinator responsible for asynchronous workflow routing and deterministic tools management.
- **Qdrant:** Serves as the high-throughput vector database managing semantic search layers, multi-collection student memories, and cross-session conceptual tracking.
- **Enkrypt AI:** Acts as an inline, synchronous safety gateway validating every model input and output against rigorous institutional guardrails including toxicity, hallucination indices, and educational bias.

2. System Architecture & Flow

The system utilizes a structured, deterministic multi-tier pipeline. Requests stream securely from the user interface down into specialized micro-agents before running memory contextualization and validation loops.



3. Frontend & Backend Core Specification

Frontend Architecture

Built completely using **Next.js, TypeScript, and Tailwind CSS** to minimize bundle size and allow seamless server-side compilation of statistical UI blocks. The interface is structurally organized into isolated modules:

- **Chat Interface:** A continuous stream-rendered terminal allowing students to run multi-turn code or conceptual queries with the AI Tutor.

- **Quiz Dashboard:** An interactive interface presenting timed markdown MCQs, instant validation scoring, and code completion modules.
- **Study Planner:** A high-fidelity scheduling dashboard mapping chronological calendars generated natively from planner agents.
- **Progress Dashboard:** Data visualization grids showcasing personalized analytics, weakness indices, and historical velocity.

Backend API Layer

Constructed on **Node.js, Express, and TypeScript**. It serves as a secure gateway, executing token-based session management, payload formatting, and logging tracking prior to delegating workflows to Mastra.

Endpoint	Method	Payload Scope	Downstream Handshake
/api/v1/chat	POST	{ query: string, sessionId: string }	Mastra Tutor Agent + Qdrant
/api/v1/quiz	POST	{ topic: string, totalQuestions: number }	Mastra Quiz Agent Engine
/api/v1/study-plan	POST	{ subjects: [], examDate: Date }	Planner Agent Optimization
/api/v1/progress	GET	Query: { studentId: string }	Progress Metrics Aggregator

4. Mastra Agent Orchestration Layer

Mastra governs the operational lifestyle of the application by running declarative state-machines and runtime tool-calling pipelines. Instead of allowing untrusted agents to trigger raw loops, Mastra acts as a deterministic boundary box.

Tutor Agent

Purpose: Conceptual decomposition and structural instruction.

Inputs: Raw student text, structural historical memory vectors.

Output: Multi-tiered educational response adapted to context levels.

Quiz Agent

Purpose: Synthesis of evaluation matrices.

Inputs: Core concept string, difficulty scalar, history parameters.

Output: Safe JSON formatted MCQs and edge-case execution tasks.

Study Planner Agent

Purpose: Dynamic operational roadmap synthesis.

Inputs: Deadlines, hourly constraints, weak nodes indices.

Output: Matrix-validated study calendars matching timeline bounds.

Progress Agent

Purpose: Deep analytical concept trace evaluation.

Inputs: Multi-turn session logs, quiz accuracy tracking histories.

Output: Statistical weak-topic listings and recommended review arrays.

5. Qdrant Memory & RAG Architecture

To facilitate contextual learning persistence, SmartTutor AI leverages isolated vector collections in **Qdrant Cloud**. This eliminates stateless agent execution drift.

Core Vector Database Collections Setup:

- **student_memory:** Dense embeddings of conversational interactions, concepts mastered, and dynamic contextual learning states.
- **quiz_history:** Structuring quantitative historical accuracies, timelines spent per node, and failure metrics.
- **weak_topics:** Isolated conceptual vectors indexing areas showing repeated sub-optimal response patterns.
- **study_plans:** Storing metadata vectors of user paths to enforce programmatic task completion monitoring.

The Execution RAG Workflow Loop

- 1. **Query Capture:** The incoming payload is passed to an optimized vector generator (e.g., text-embedding-3-small).
- 2. **Semantic Probe:** Qdrant processes a high-velocity k-NN vector search against the user's isolated collection segment.
- 3. **Inject Context:** Relevant historical contexts match weights and are appended as immutable system parameters to the Mastra payload.
- 4. **Synthesize:** The corresponding AI Agent evaluates the context-augmented frame, ensuring exact continuation of the educational journey.

6. Enkrypt AI Guardrails & Database Mapping

To meet enterprise educational standards, all input-output vectors interact synchronously with the **Enkrypt AI Core API Gateway** to eliminate malicious model trends and behavioral drift before UI injection.

- **Hallucination Scrubber:** cross-checks historical documents and memory arrays to prevent factual errors.
- **Educational Safety Index:** Blocks out non-scholastic vectors, toxicity patterns, and unexpected political biases.
- **Quality Validation Matrix:** Emits telemetry metrics evaluating the precision, length, and technical accuracy of generated data.

Relational Storage Framework (PostgreSQL)

While Qdrant records the un-structured semantic memories, core structural user states, financial records, and metrics are committed natively to **PostgreSQL**.

Table Name	Primary & Foreign Keys	Critical Fields / Telemetry Payload
users	user_id (PK)	name, email, role_enum, created_at, session_token
quiz_results	quiz_id (PK), user_id (FK)	raw_score, total_answered, concept_tag, metadata_json
study_plans	plan_id (PK), user_id (FK)	schedule_matrix, active_status, target_milestone_date
progress_reports	report_id (PK), user_id (FK)	weakness_array, improvement_index, aggregate_velocity

7. Infrastructure, Security & Scalability Blueprint

The production deployment footprint is built around a modern distributed system infrastructure pattern. Frontends deploy to edge nodes via **Vercel**, backends scale via horizontally orchestrated stateless containers, and standard relational transactions settle in a highly available **PostgreSQL Cluster**.

Scalability and Performance Matrix

- **Distributed Vector Topologies:** Uses Qdrant Cloud horizontal shading patterns to ensure low-latency operations under heavy indexing loads.
- **Stateless APIs:** Decoupling session verification via signed JWT structures, ensuring servers require zero localized variable caching.
- **Async Queue Buffers:** Long-running study map Generations are processed out-of-band via background worker engines to keep API responses below the critical 5-second window.

Security and Governance Framework

- **Data Transit Guarding:** Strict enforcement of TLS 1.3 for all external communication and AES-256 block encryption for stationary table items.
- **Access Enforcement:** Granular control protocols utilizing Role-Based Access Controls (RBAC) to completely segregate teacher analytics spaces from standard student environments.

8. Future Strategy & Success Framework

To ensure ongoing development viability after the initial launch phase, the platform's evolutionary vector follows a modular feature expansion layout:

Phase Tracking	Feature Focus	Downstream Success Metric Target
Phase 2	Voice Tutoring Integration Module	Perceptual Interaction Latency < 850ms
Phase 3	PDF and Textbook Semantic Ingestion Assistant	Chunk Processing Speeds > 150 pages/min
Phase 4	Native Multi-Language Academic Ingestion	Localization Scoring Balance Matrix > 98%
Phase 5	Automated AI Career Pathing & Prediction	Structural Student Alignment Accuracy > 92%

Conclusion

SmartTutor AI transforms modern learning from standard stateless chat interfaces into an institutional-grade, persistent digital companion ecosystem. By implementing a strict execution topology through the combination of

Mastra orchestration, **Qdrant** vector intelligence, and **Enkrypt AI** validation loops, the design layout guarantees a high-performance, safe, and robust production-ready platform.